Module -1 DAA

Q1. What is an Algorithm?

An **algorithm** is a sequence of steps to solve a problem. It acts like a set of instructions on how a program should be executed.
There is no fixed structure of an algorithm.
Design and Analysis of Algorithms covers the concepts of designing an algorithm to solve various problems in computer science and information technology and analyzing the complexity of these designed algorithms.

1) To understand the basic idea of the problem.
2) To find an approach to solve the problem.
3) To improve the efficiency of existing techniques.
4) To understand the basic principles of designing the algorithms.
5) To compare the performance of the algorithm with respect to other techniques.

Property's of ageratums are : Finiteness, Definiteness, Input, Output, Effectiveness

Q2 Difference between program and algorithm?

An algorithm is a step by step procedure to solve a specific problem. } An program is also a step by step procedure to solve a specific problem

- An algorithm is an abstract concept. Where as , a program is a concrete implementation of the algorithm.
- An algorithm can be written in any language, while a program must be written using a programming language only.
- An algorithm is developed during the design phase and a program is developed during the development phase.
- An algorithm does not depend on the hardware and operating system while a program depends upon them.
- An algorithm is always analyzed, while a program is tested.

Q3 performance analysis in algorithms?

To compare how efficiently one algorithm performs against another for the same problem, we measure its **complexity** in two ways:

1. **Time Complexity** – The amount of time an algorithm requires for execution.
2. **Space Complexity** – The amount of memory an algorithm requires for execution.

Time Complexity

The **time complexity** of an algorithm represents the amount of time it takes to execute based on input size.
There are three types of time complexity:

1. **Best Case (Ω)** – The minimum time required for execution (fastest scenario).
2. **Worst Case (O)** – The maximum time required for execution (slowest scenario).
3. **Average Case (Θ)** – The expected time required on average inputs.

Example: Linear Search
Consider an array: **[1, 2, 3, 4, 5, 6]**

- **Key = 1 → O(1)** (Best case, found immediately).
- **Key = 6 → O(n)** (Worst case, found at the last position).
- **Key = 3 → O(n-1)** (Somewhere in between).

Steps to Calculate Time Complexity

1. **Identify the number of operations**
   - Example: Variable declaration → **O(1)**
   - Example: Loops → Consider number of iterations.
2. **Ignore constants**
   - Example: **2n, n/2, n+1** all simplify to **O(n)**.
3. **Consider the highest-order term**
   - Example: Nested loops → Multiply iterations.
   - Example:

```
for(i = 1; i <= n; i++) {
    for(j = 1; j <= i; j++) {
        // Code execution
    }
}
```

Here, the **time complexity** is **$O(n^2)$** (since the inner loop runs i times for each i).

Q4. What are Time Complexity Types & Their Characteristics?

1. **Constant Time Complexity – O(1)**
   - The number of steps required to execute the problem remains **constant**.
   - **Time does not change** even if the input size increases.
   - **Example:** All arithmetic operations (addition, subtraction, multiplication, division).
2. **Linear Time Complexity – O(n)**
   - The number of steps required **depends on the input size**.
   - If the input size increases, the execution time **increases proportionally**.
   - **Example:** Linear Search.
3. **Logarithmic Time Complexity – O(log n)**
   - The problem is divided into **smaller parts** after each step.
   - The number of steps required **reduces by half** with each iteration.
   - **Example:** Binary Search.
4. **O(n log n) Time Complexity – O(n log n)**
   - The problem is divided into smaller parts, and after solving each part, they are **combined**.
   - The number of steps required reduces **log n** times while solving the problem.
   - **Example:** Merge Sort.
5. **Quadratic Time Complexity – $O(n^2)$**
   - The number of steps required **grows exponentially** as input size increases.
   - For input size **n**, the number of steps required is **n^2**.
   - **Very slow** for large inputs.
   - **Example:** Bubble Sort.

Amount of space an algorithm requires to complete its execution is called space complexity.

Q5. What is Asymptotic Notations & Complexity Analysis? (10marks)

1. **Asymptotic Notations** measure how execution time increases as input size grows.
2. **Big-O (O)** – Represents the **worst-case** time complexity (upper bound).
   - It defines the **maximum** time an algorithm takes.
   - **Example:** Bubble Sort → **$O(n^2)$**
3. **Big-Omega (Ω)** – Represents the **best-case** time complexity (lower bound).
   - It defines the **minimum** time an algorithm takes.
   - **Example:** Binary Search → **Ω(log n)**
4. **Big-Theta (Θ)** – Represents the **average-case** time complexity (tight bound).
   - It defines the **expected** time an algorithm takes.
   - **Example:** Merge Sort → **Θ(n log n)**
5. **Big-O Notation Formal Definition:**
   - If **$f(n) \leq c * g(n)$** for all **$n \geq n_0$**, then **$f(n) \in O(g(n))$**

6. **Big-Ω Notation Formal Definition:**
   - If $f(n) \geq c * g(n)$ for all $n \geq n_0$, then $f(n) \in \Omega(g(n))$
7. **Big-Θ Notation Formal Definition:**
   - If $c_1 * g(n) \leq f(n) \leq c_2 * g(n)$ for all $n \geq n_0$, then $f(n) \in \Theta(g(n))$
8. **Comparison of Growth Rates:**
   - $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$
9. **Practical Examples:**
   - **Binary Search: O(log n), Ω(1), Θ(log n)**
   - **Linear Search: O(n), Ω(1), Θ(n)**
   - **Merge Sort: O(n log n), Ω(n log n), Θ(n log n)**
   - **Bubble Sort: $O(n^2)$, Ω(n), $\Theta(n^2)$**
10. **Key Takeaway:**

- **Big-O (Worst Case) → Maximum time taken.**
- **Big-Ω (Best Case) → Minimum time taken.**
- **Big-Θ (Average Case) → Expected time taken.**

Problems & Ans:

1. Prove that $1000.n^2 + 1000.n = O(n^2)$
2. Prove that $2n + 10 = O(n)$
3. Prove that :$3n^3 + 20n^2 + 5 = O(n^3)$
4. Prove that : $10n + 500 = O(n)$
5. Prove that $5n^2 + 2n\text{-}3 \in \Omega(n^2)$

1. Prove that 1000n^2 + 1000n = O(n^2)

We need to find constants c and n0 such that:
1000n^2 + 1000n ≤ c · n^2, for all n ≥ n0.
Rewrite the expression:
1000n^2 + 1000n = 1000n^2 (1 + 1/n)
For n ≥ 1, note that:
1 + 1/n ≤ 2
Therefore:
1000n^2 (1 + 1/n) ≤ 1000n^2 · 2 = 2000n^2

Choose c = 2000 and n0 = 1.
Hence, 1000n^2 + 1000n = O(n^2).

---

2. Prove that 2n + 10 = O(n)

We need to find constants c and n0 such that:
2n + 10 ≤ c · n, for all n ≥ n0.
Rewrite the expression:
2n + 10 = n (2 + 10/n)
For n ≥ 10, note that:
10/n ≤ 1 ⇒ 2 + 10/n ≤ 3
Thus:
2n + 10 ≤ 3n
Choose c = 3 and n0 = 10.
Therefore, 2n + 10 = O(n).

---

3. Prove that $3n^3 + 20n^2 + 5 = O(n^3)$

We need to find constants c and n0 such that:

$3n^3 + 20n^2 + 5 \leq c \cdot n^3$, for all $n \geq n0$.

Rewrite the expression:

$3n^3 + 20n^2 + 5 = n^3 (3 + 20/n + 5/n^3)$

For $n \geq 5$, note that:

$20/n \leq 4$ and $5/n^3 \leq 1$

So, $3 + 20/n + 5/n^3 \leq 3 + 4 + 1 = 8$

Thus:

$3n^3 + 20n^2 + 5 \leq 8n^3$

Choose c = 8 and n0 = 5.

Therefore, $3n^3 + 20n^2 + 5 = O(n^3)$.

---

4. Prove that $10n + 500 = O(n)$

We need to find constants c and n0 such that:

$10n + 500 \leq c \cdot n$, for all $n \geq n0$.

Rewrite the expression:

$10n + 500 = n (10 + 500/n)$

For $n \geq 50$, note that:

$500/n \leq 10 \Rightarrow 10 + 500/n \leq 20$

Thus:

$10n + 500 \leq 20n$

Choose c = 20 and n0 = 50.

Therefore, $10n + 500 = O(n)$.

---

5. Prove that $5n^2 + 2n - 3 = \Omega(n^2)$

We need to find constants c and n0 such that:

$5n^2 + 2n - 3 \geq c \cdot n^2$, for all $n \geq n0$.

Rewrite the expression:

$5n^2 + 2n - 3 = n^2 (5 + 2/n - 3/n^2)$

For $n \geq 1$, note that the terms 2/n and $-3/n^2$ become less significant.

In particular, for $n \geq 1$: $5 + 2/n - 3/n^2 \geq 5$.

Thus:

$5n^2 + 2n - 3 \geq 5n^2$

Choose c = 5 and n0 = 1.

Therefore, $5n^2 + 2n - 3 = \Omega(n^2)$.